

CROSS DISCIPLINARY PROJECT

Project Handbook

Introduction

Dear students,

This handbook defines the organization, **expectations, deliverables, and evaluation process** for your **Term 2 Cross-Disciplinary Project**.

The project involves collaboration between the following courses:

- ✓ Back End Development
- ✓ Database Administration
- ✓ HCI (Human Computer Interaction)
- ✓ Software Engineering

The project is conducted in groups and allows you to apply concepts from multiple courses within a single shared project context.

Note that this is **not a capstone project**: each course will evaluate the parts of the project related to its own subject and learning outcomes.

To succeed in this project:

- start early,
- communicate regularly within your group,
- validate your work frequently **with all lecturers**,
- and **ensure all course requirements are considered**.

Good organization and teamwork are essential for success.

Good luck everyone!

Agenda

The project is divided into **4 key phases**: Ideation > proposal > Implementation > Defense

WEEK	PHASE	DESCRIPTION
Week 3	IDEATION	Propose initial project ideas, features, and target users. HCI lecturers provide early feedback on feasibility and direction.
Week 6	PROPOSAL	Submit the project proposal. All Lecturers validate or reject the proposal according to their course requirements.
Week 6...10	IMPLEMENTATION	Teams design, develop, test, and document the project
Week 11	DEFENSE & DELIVERABLES	Students present the project and submit final deliverables . Each lecturer evaluates their own course outcomes.

Project Team Size

- Each team shall be composed of 3 members.
- No cross-group teams
- The last group can be composed of 2 members

Project Proposal

Your group must prepare a **project proposal** explaining:

- the project idea,
- the target users,
- the main features,
- and how the project matches the 4 courses requirements

Each lecturer **will validate whether your proposal satisfies the requirements** of their course.

A proposal may be:

- accepted,
- accepted with modifications,
- or rejected if important requirements are missing.

Project Requirements Per Course

Each course has its own expectations and validation criteria.

Your project proposal and final implementation must **satisfy the requirements defined by each course.**

Human Computer Interaction

- ✓ Target users clearly identified
- ✓ UX research conducted (interviews, surveys, observations, etc.)
- ✓ User pain points analyzed
- ✓ At least one persona created
- ✓ User stories defined for main features
- ✓ User flow or journey documented
- ✓ Wireframes or mockups created before implementation
- ✓ Interface designed using usability and consistency principles
- ✓ Clear navigation and user feedback implemented

Software Engineering

- ☐ Functional and non-functional requirements identified (Main Features in Applications)
 - Including the Development Methodology (SCRUM, Waterfall...etc.)
- ✓ UML diagrams created where required
 - 4 Diagram (Use case, Activity, Class and Sequence)
 - Including descriptions for each diagram
- ✓ Team responsibilities organized per user stories
- ✓ Development planning demonstrated

Backend Development

- ✓ REST API implemented using Node.js and Express.js
- ✓ API routes separated properly (routes/controllers/services/repositories)
- ✓ Authentication implemented using JWT tokens
- ✓ Protected routes requiring authentication

- ✓ Authorization / privilege management implemented (roles or permissions)
- ✓ API connected to a database
- ✓ CRUD operations implemented for main resources
- ✓ API tested using Postman or Swagger UI or ApiDog
- ✓ Swagger API documentation available with testable requests
- ✓ Proper HTTP status codes used
- ✓ Environment variables stored in. env
- ✓ Error handling middleware implemented
- ✓ Frontend integration with ReactJS

Database Administration

- ✓ Database implemented using MySQL Database (Oracle Database, PostgreSQL, etc.)
- ✓ Database Design and Analysis: ERD and Relational Model (RM) designed appropriately
 - ✓ Primary keys and foreign keys correctly implemented
 - ✓ Relationships between entities properly designed
 - ✓ Constraints implemented where relevant
- ✓ SQL scripts provided for: table creation, insertion, and queries
- ✓ Sample data available
- ✓ Database connected successfully to the application
- ✓ Meaningful queries demonstrated
- ✓ User access and control management: **User Privilege and Roles**
- ✓ **Backup and recovery** mechanisms (export scripts provided if requested)
- ✓ Query optimization techniques: **indexing** implemented properly
- ✓ **Data encryption** implemented where necessary